

Production deploy (my-chat)

Status

Sprint 5 is DONE. Prod is live:

- Domain: **https://beep.ru** (VPS IP **87.228.112.251**)
- SSH: host alias **ssh my-chat** (see local **~/.ssh/config**)
- CD: push to **main** → GitHub Actions → SSH pull/build/up + **nginx -s reload**

Do **not** rebuild the deploy stack from scratch. Extend existing files. Sprint 6 adds VAPID/webpush wiring on top of this stack.

Known gotchas: **docs/known-limitations-sprint-5.md** (nginx **set** before **rewrite+break**; certbot in **.dockerignore**; nginx reload after compose; no **www.beep.ru**; prod seed was manual).

Architecture

```
Internet → Nginx :80/:443 → main-service:8080 | auth-proxy:33081
Internal only: postgres, notification-worker, message-expirer [,
prometheus]
```

Path routing:

URL	Backend
/api/	main-service:8080
/ws/	main-service:8080 (WebSocket upgrade)
/auth/	auth-proxy:33081
/health	main-service:8080

Repo layout:

- **deploy/prod/docker-compose.prod.yml**
- **deploy/prod/nginx/**
- **deploy/prod/init-ssl.sh** (first TLS / renew helper)
- **deploy/prod/.env.example** and root **.env.example** → real secrets only in **/opt/my-chat/.env** on VPS
- **configs/config.*.prod.yaml**
- **.github/workflows/ci.yml, cd.yml**

Secrets (never commit)

Where	What
VPS <code>/opt/my-chat/.env</code>	<code>POSTGRES_PASSWORD</code> , <code>JWT_SECRET</code> , <code>DOMAIN</code> , <code>VAPID_*</code> , <code>PUSH_PROVIDER</code> , metrics auth
GitHub Secrets	<code>VPS_HOST</code> , <code>VPS_USER</code> , <code>VPS_SSH_KEY</code>
Git	only <code>.env.example</code> with placeholders

Ignore: `.env`, `deploy/prod/certbot/`, `deploy/prod/nginx/ssl/`.

VAPID (Sprint 6): keys are already on the VPS `.env` (`VAPID_PRIVATE_KEY`, `VAPID_PUBLIC_KEY`, `VAPID_SUBJECT`). Placeholders exist in `.env.example`. Until §7–15: notification-worker prod config is still **provider: noop** — do not claim push works in prod. After webpush code: wire env into compose for worker + main-service, set **provider: webpush**, reload nginx if needed.

DNS (Selectel)

1. Zone for `beep.ru` — A @ → VPS IP (done for Sprint 5).
2. `www` is **not** configured (known limitation).
3. Verify: `dig beep.ru A` → `87.228.112.251`.

One-time VPS prep

Already done for Sprint 5. Reference: `docs/sprint-5-checklist.md` §7. Summary if rebuilding a host:

1. Docker Engine + Compose plugin.
2. User `deploy` in group `docker` (prefer CD as `deploy`, not root).
3. ufw: allow 22, 80, 443 only.
4. Repo at `/opt/my-chat`.
5. `/opt/my-chat/.env` from `.env.example`.
6. TLS: `bash deploy/prod/init-ssl.sh [--staging]` then real cert; nginx only public 80/443.

Deploy commands (on VPS)

Prefer CD via `main`. Manual:

```
cd /opt/my-chat
git pull origin main
docker compose -f deploy/prod/docker-compose.prod.yml build
docker compose -f deploy/prod/docker-compose.prod.yml up -d --remove-orphans
docker exec my-chat-nginx-prod nginx -s reload # volume-mounted conf may not apply otherwise
docker compose -f deploy/prod/docker-compose.prod.yml ps
docker image prune -f
```

Smoke (prod)

```
curl -fsS "https://beep.ru/health"
# Auth (username/password – Sprint 6; not user_id):
curl -sS -X POST "https://beep.ru/auth/api/v1/auth/login" \
  -H 'Content-Type: application/json' \
  -d '{"username":"<user>","password":"<pass>"}'
# Register (main-service):
# POST https://beep.ru/api/v1/users/register
# Then: GET https://beep.ru/api/v1/me/unread-count with Bearer
# wss://beep.ru/ws/connect?token=...
```

Also: browser without TLS warnings; postgres/8080 **not** on public IP; HSTS / HTTP→HTTPS 301.

Prod may lack seed users — use register or manual SQL ([docs/known-limitations-sprint-5.md](#) §5). Mobile/Debug still on `user_id` until Sprint 6 client items — curl for auth smoke.

Agent rules

- Prefer editing repo files under `deploy/prod/`, prod configs, workflows; do not invent alternate deploy stacks.
- Never write real secrets into git or chat logs if avoidable; use placeholders.
- Align checklist updates with skill `sprint-work` (Sprint 6 infra = §15; do not reopen Sprint 5 as incomplete).
- Local stack is separate — use skill `local-dev` for `task local:*`.
- After compose changes that only touch mounted nginx conf, ensure **nginx reload** (CD does this; manual deploys must too).